



Tecnológico Nacional de México

Instituto Tecnológico de Chiná

Nombre de la Carrera:

INGENIERÍA INFORMÁTICA

Nombre de la Asignatura:

PROGRAMACION AMBIENTE CLIENTE SERVIDOR

Semestre y grupo:

Séptimo Semestre 7AI

Actividad:

U1 Actividad 1

Nombre del
estudiante:

**Edgar Ernesto Ek
Koh**

**Chi Chablé Miguel
Eduardo**

**Aguilar León Aaron
Jesús Eloy**

PROG-AMB-CL-SRV_UNIDAD1-ACTIVIDAD1

Nombre del Profesor:
Jenner Noel che mendez

Lugar y Fecha:

CHINÁ, CAMPECHE, 1 DE OCTUBRE DEL 2026



Objetivo de la práctica.

Básicamente, el objetivo de esta práctica es entender cómo se comunican un cliente y un servidor en la vida real, analizando de primero los elementos que viajan en una conversación HTTP, como sus métodos, cabeceras, códigos de estado y tiempos.

La idea clave es comprobar que no importa realmente la herramienta que utilices ya sea el navegador web, la terminal con curl o un cliente especializado como Postman, ya que al final todas se comunican con el mismo servidor respetando el mismo protocolo y contrato.

Con esto logramos conectar el funcionamiento de las peticiones web con las operaciones de bases de datos y el enfoque de diseño API-first.

Concepto de petición HTTP.

Bueno una petición HTTP es básicamente el mensaje o la solicitud que manda el cliente como en este caso es tu navegador, Postman o la terminal al servidor.

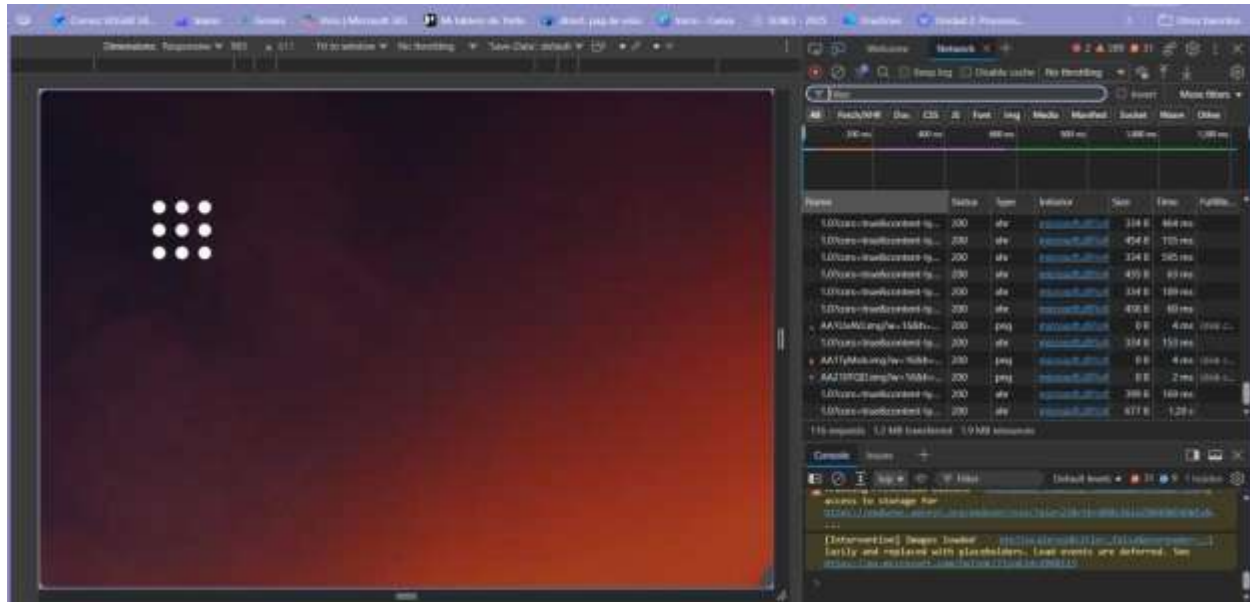
Todo esto para pedirle algo, ya sea consultar una página, traer datos o mandar información nueva. Va empaquetada con cosas clave como el método, un GET si solo quieres leer o un POST si vas a crear algo, la ruta o URL exacta del recurso.

Concepto de respuesta HTTP.

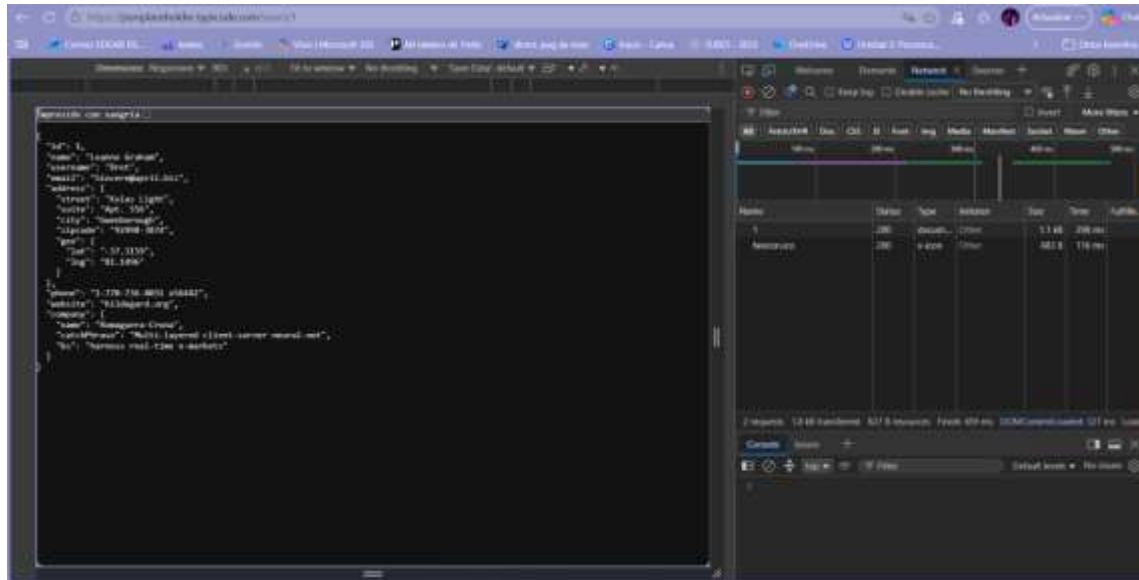
La respuesta HTTP es lo que el servidor te regresa de vuelta después de procesar lo que le pediste.

Esta respuesta incluye un código de estado (como el clásico 200 OK si todo salió bien, un 201 Created si guardaste algo, o un 404 si no encontró lo que buscabas), encabezados que indican el tipo de contenido que te manda (por ejemplo, application/json) y el cuerpo llamado body con la información o los datos listos para que el cliente los use.

Captura de Network del navegador.



Identificación del método HTTP.



Identificación del código de estado.

The screenshot shows the Chrome DevTools interface. The **Network** panel is active, displaying a list of requests. The first request, **favicon.ico**, is selected. The **Headers** sub-panel is open, showing the following details:

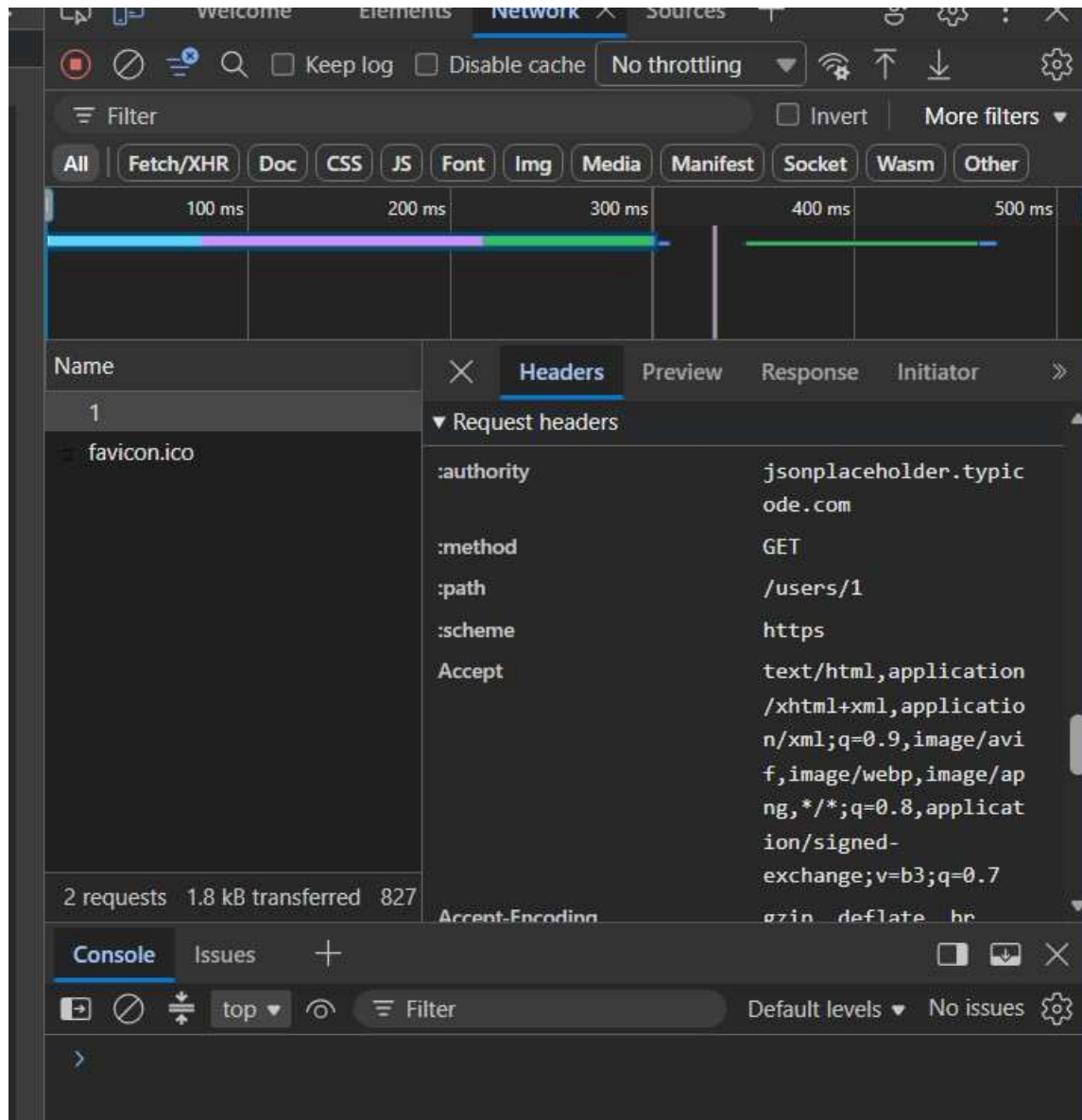
- Request URL:** `https://jsonplaceholder.typicode.com/users/1`
- Request Method:** `GET`
- Status Code:** `200 OK`
- Remote Address:** `104.21.59.19:443`
- Referrer Policy:** `strict-origin-when-cross-origin`

Below the request details, the **Response headers** section is visible, showing:

- Access-Control-Allow-Credentials:** `true`
- Age:** `5351`
- Alt-Svc:** `h3=":443"`

The **Console** panel is also visible at the bottom, showing no issues.

Identificación de Request Headers.



Identificación de Response Headers.



Network tab interface showing a timeline of network requests and the details of the first request (1) in the Headers section.

Timeline:

- 100 ms
- 300 ms
- 400 ms
- 500 ms

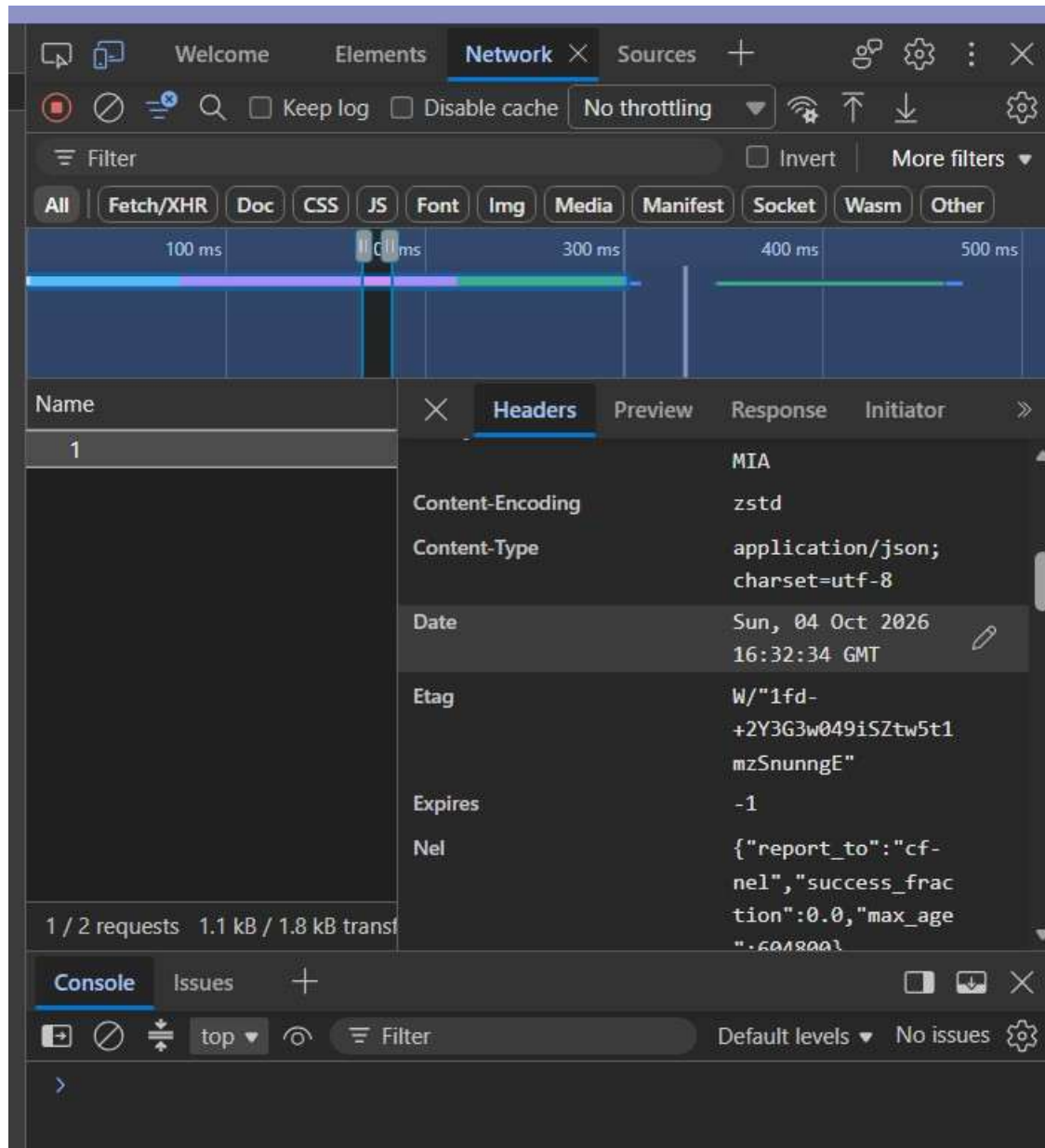
Request 1 Details:

Name	Value
Access-Control-Allow-Credentials	true
Age	5351
Alt-Svc	h3=":443"; ma=86400
Cache-Control	max-age=43200
Cf-Cache-Status	HIT
Cf-Ray	a45595576a8ef471-MIA
Content-Encoding	zstd
Content-Type	application/json; charset=utf-8

1 / 2 requests 1.1 kB / 1.8 kB transferred

Console:

Issues: top Filter Default levels No issues





Network tab interface showing a list of requests and the details of the selected request (1).

Network tab: Welcome, Elements, Network, Sources. Filter: All, Fetch/XHR, Doc, CSS, JS, Font, Img, Media, Manifest, Socket, Wasm, Other. No throttling. Invert. More filters.

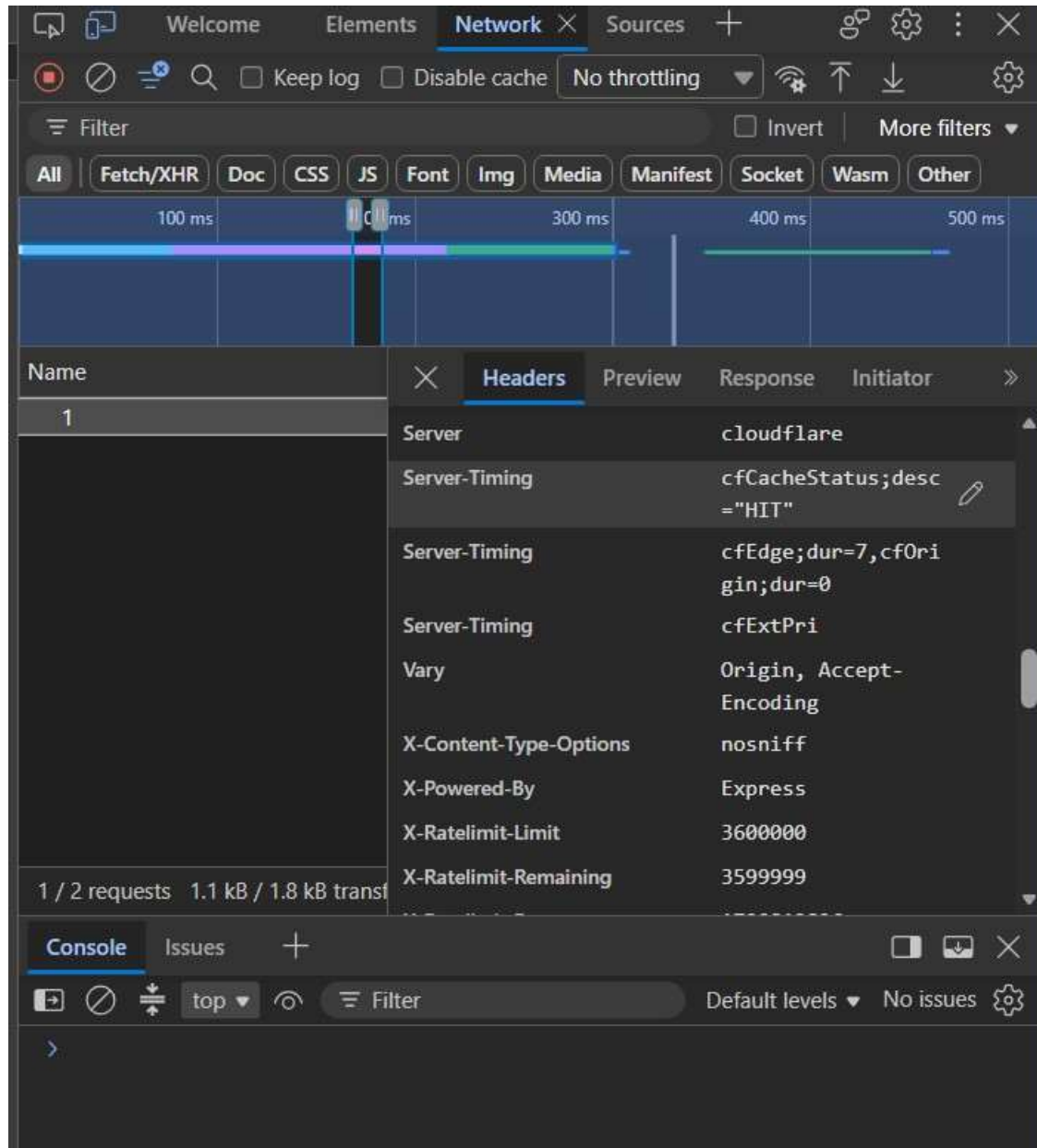
Timeline: 100 ms, 200 ms, 300 ms, 400 ms, 500 ms.

Request 1 details:

Name	Headers	Preview	Response	Initiator
1	Date: Sun, 04 Oct 2026 16:32:34 GMT Etag: W/"1fd-+2Y3G3w049i9mzSnunngE" Expires: -1 Nel: {"report_to": "cf-nel", "success_fraction": 0.0, "max_age": 604800} Pragma: no-cache Priority: u=0,i Report-To: {"group": "cf-nel", "max_age": 604800, "report_to": "cf-nel"}			

1 / 2 requests 1.1 kB / 1.8 kB transferred

Console tab: Issues. Filter. Default levels. No issues.



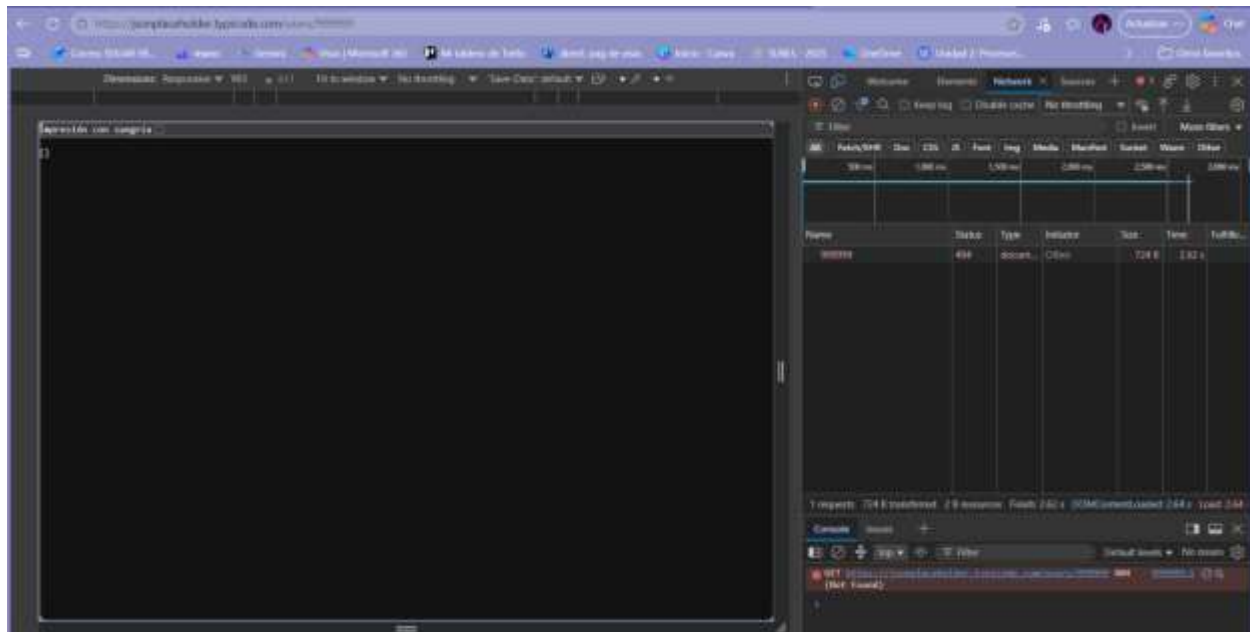
Captura del cuerpo JSON.

The screenshot shows the Chrome DevTools Network tab. The top bar includes tabs for Welcome, Elements, Network, Sources, and a plus icon. Below the tabs are checkboxes for Keep log, Disable cache, and No throttling. A filter bar shows 'All' selected, with other filters like Fetch/XHR, Doc, CSS, JS, Font, Img, Media, Manifest, Socket, Wasm, and Other. A timeline bar at the top shows a single request taking approximately 100 ms. The main pane displays the response of the first request, which is a JSON object. The response is shown in the 'Response' tab, with other tabs like Headers, Preview, and Initiator visible. The JSON data is as follows:

```
{
  "id": 1,
  "name": "Leanne Graham",
  "username": "Bret",
  "email": "Sincere@april.biz",
  "address": {
    "street": "Kulas Light",
    "suite": "Apt. 556",
    "city": "Gwenborough",
    "zipcode": "92998-3874",
    "geo": {
      "lat": "-37.3159",
      "lng": "81.1496"
    }
  },
  "phone": "1-770-736-8031 x56442",
  "website": "hildegard.org",
  "company": {
    "name": "Romaguera-Crona",
    "catchPhrase": "Multi-layered client-serv",
    "bs": "harness real-time e-markets"
  }
}
```

At the bottom, the status bar shows '1 / 2 requests' and '1.1 kB / 1.8 kB tra'. The bottom of the window shows the Console and Issues tabs.

15. Ejercicio 404



Ejecución de curl.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\kohed> curl.exe https://jsonplaceholder.typicode.com/users/1
{
  "id": 1,
  "name": "Leanne Graham",
  "username": "Bret",
  "email": "Sincere@april.biz",
  "address": {
    "street": "Kulas Light",
    "suite": "Apt. 556",
    "city": "Gwenborough",
    "zipcode": "92998-3874",
    "geo": {
      "lat": "-37.3159",
      "lng": "81.1496"
    }
  },
  "phone": "1-770-736-8031 x56442",
  "website": "hildegard.org",
  "company": {
    "name": "Romaguera-Crona",
    "catchPhrase": "Multi-layered client-server neural-net",
    "bs": "harness real-time e-markets"
  }
}
PS C:\Users\kohed> |
```

Ejecución de curl -i.



```
Windows PowerShell

"attribPress": "Multi-layered client-server neural-net",
"ba": "harvest real-time e-markets"
}

PS C:\Users\kalede> curl.exe -o https://joeplacasholder.typicode.com/users/1
HTTP/1.1 200 OK
Date: Sat, 08 Oct 2026 17:20:16 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 899
Connection: keep-alive
Access-Control-Allow-Credentials: true
Cache-Control: max-age=31288
etag: W/"1d2">303604152b6tlnGmng5"
expires: -1
pragma: no-cache
vary: Origin, Accept-Encoding
x-content-type-options: nosniff
x-powered-by: Express
x-ratelimit-limit: 1000000
x-ratelimit-remaining: 1000000
x-ratelimit-reset: 1796031826
Age: 8275
Accept-Ranges: bytes
cf-cache-status: HIT
Bul: [{"report_to":"cf-eval","success_fraction":0.0,"max_age":600000}]
Report-To: [{"group":"cf-eval","max_age":600000,"endpoints":[{"url":"https://a.nel.cloudflare.com/report/v4?rt=cf-eval&token=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9IiwiaXNjaWkiOiJkaWZlIiwiaWF0IjoiMTY5NjA2MjY4MjY1IiwiaWZlbnR1eSI6ImVudC50eXNjaWki"}]}]
Server: cloudflare
CF-Ray: a455d3b18070a1-016
alt-svc: h3="443"; ma=86400

{
  "id": 1,
  "name": "Leanne Graham",
  "username": "Bret",
  "email": "lsgreen@qpril.biz",
  "address": {
    "street": "Wiles Light",
    "suite": "Apt. 380",
    "city": "Duckborough",
    "zipcode": "82890-1070",
    "geo": {
      "lat": "-37.3150",
      "lng": "82.1490"
    }
  },
  "phone": "1-778-710-8931 x56AA2",
  "website": "kildagart.org",
  "company": {
    "name": "Reapace-Cross",
    "catchPhrase": "Multi-layered client-server neural-net",
    "ba": "harvest real-time e-markets"
  }
}

PS C:\Users\kalede>
```

Ejecución de curl -v.

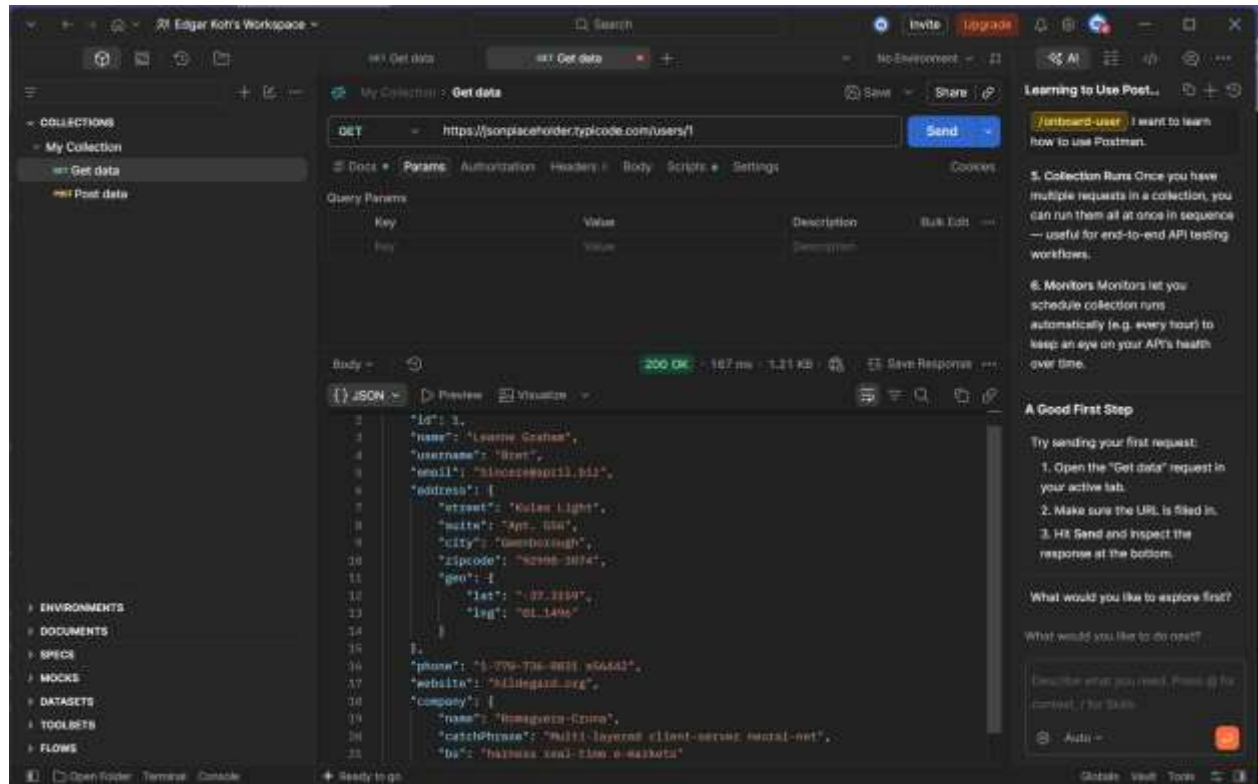
```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\kalede> curl.exe -v https://joeplacasholder.typicode.com/users/1
* Host joeplacasholder.typicode.com:443 was resolved.
* IP*: 100.21.88.10, 172.67.207.131
* Trying 100.21.88.10...
* Connected to joeplacasholder.typicode.com (100.21.88.10 port 443)
* ALPN: curl offers h2/h3
* ALPN: server accepted h2/h3
* Established connection to joeplacasholder.typicode.com (100.21.88.10 port 443) from 102.168.1.185 port 52118
* using HTTP/2.0
* GET /users/1 HTTP/1.1
* Host: joeplacasholder.typicode.com
* User-Agent: curl/8.21.0
* Accept: */*
*
* Request completely sent off
* channel: reuse party requests renegotiation
* channel: renegotiating SSL/TLS connection
* channel: SSL/TLS connection renegotiated
* HTTP/1.1 200 OK
Date: Sat, 08 Oct 2026 17:21:41 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 899
Connection: keep-alive
Access-Control-Allow-Credentials: true
Cache-Control: max-age=31288
etag: W/"1d2">303604152b6tlnGmng5"
expires: -1
pragma: no-cache
vary: Origin, Accept-Encoding
x-content-type-options: nosniff
x-powered-by: Express
x-ratelimit-limit: 1000000
x-ratelimit-remaining: 1000000
x-ratelimit-reset: 1796031826
Age: 8276
Accept-Ranges: bytes
cf-cache-status: HIT
Bul: [{"report_to":"cf-eval","success_fraction":0.0,"max_age":600000}]
Report-To: [{"group":"cf-eval","max_age":600000,"endpoints":[{"url":"https://a.nel.cloudflare.com/report/v4?rt=cf-eval&token=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9IiwiaXNjaWkiOiJkaWZlIiwiaWF0IjoiMTY5NjA2MjY4MjY1IiwiaWZlbnR1eSI6ImVudC50eXNjaWki"}]}]
Server: cloudflare
CF-Ray: a455d3b18070a1-016
alt-svc: h3="443"; ma=86400
{
  "id": 1,
  "name": "Leanne Graham",
  "username": "Bret",
  "email": "lsgreen@qpril.biz",
  "address": {
    "street": "Wiles Light",
    "suite": "Apt. 380",
    "city": "Duckborough",
    "zipcode": "82890-1070",
    "geo": {
      "lat": "-37.3150",

```



Captura de petición realizada mediante cliente API.



Comparación entre las tres herramientas.



Resultados de GET y POST

The screenshot shows the Postman interface with a GET request to `https://jsonplaceholder.typicode.com/users/1`. The response is a 200 OK status with a response time of 167 ms and a body size of 1.21 KB. The response body is displayed in JSON format, showing user details for the first user in the database.

```
{
  "id": 1,
  "name": "Leanne Graham",
  "username": "Sret",
  "email": "Sincere@april.biz",
  "address": {
    "street": "Kulas Light",
    "suite": "Apt. 558",
    "city": "Gwenborough",
    "zipcode": "92998-3874",
    "geo": {
      "lat": "-37.3159",
      "lng": "81.1496"
    }
  },
  "phone": "1-770-736-8831 x56442",
  "website": "hildegard.org",
  "company": {
    "name": "Romaguera-Crona",
    "catchPhrase": "Multi-layered client-server neural-net",
    "bs": "harness real-time e-markets"
  }
}
```

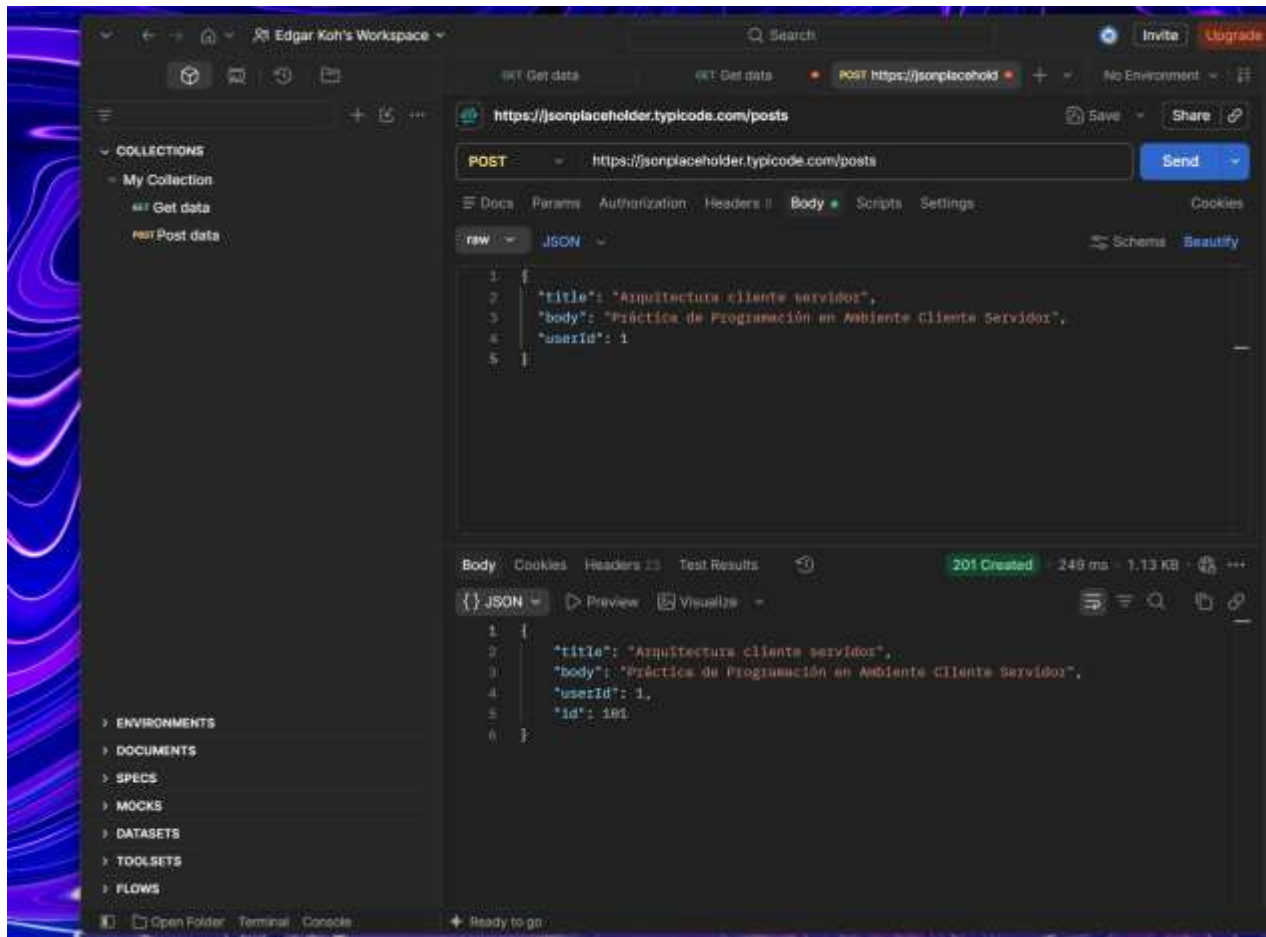


Tabla comparativa.

CARACTERISTICAS	NAVEGADOR	CURL	Cliente API (Postman)
Método HTTP	GET	GET	GET
Código	200	200	200
Formato	JSON	JSON	JSON
Tiempo visible ⁹	SI	SI	SI
Fácil para el usuario	ALTA	BAJA	MEDIA
Fácil para el desarrollador	ALTA	ALTA	MUY ALTA
Permite automatización	LIMITADA	MUY ALTA	MEDIA



Conclusiones.

En conclusión, esta práctica me sirvió bastante para dejar de ver el protocolo HTTP y la arquitectura cliente-servidor como simples teorías de salón y entender cómo funcionan en la vida real. Comprobé que no importa si mandas una petición desde el navegador, la terminal con curl o una herramienta especializada como Postman, ya que al final el servidor recibe exactamente la misma estructura de datos y te devuelve el mismo resultado mientras hables su mismo idioma. Esto te abre los ojos como desarrollador para darte cuenta de que cualquier botón que le piques a una aplicación moderna en el futuro se traduce en una solicitud HTTP por debajo, lo cual es la base para diseñar buenas APIs y separar el trabajo entre el frontend y el backend.



- 1.- **¿Quién es el cliente durante una petición realizada desde Chrome?** El propio navegador Google Chrome, que es el que arma y manda la solicitud para pedir la información.
- 2.- **¿Quién es el cliente cuando utilizamos curl?** La herramienta de la terminal curl, que actúa como nuestro cliente de línea de comandos para hablarle al servidor.
- 3.- **¿Qué función tiene el servidor?** Recibir lo que le pedimos, procesar la lógica o consultar la base de datos, y regresarnos una respuesta con los datos que solicitamos.
- 4.- **¿Qué diferencia existe entre petición y respuesta?** La petición es lo que manda el cliente pidiendo algo (con su método y URL), y la respuesta es lo que el servidor le devuelto de regreso (con el estado y los datos).
- 5.- **¿Qué significa el método GET?** Básicamente le estamos diciendo al servidor: "entrégame o consúltame este recurso".
- 6.- **¿Qué significa POST?** Es para avisarle al servidor que queremos crear o registrar un recurso nuevo mandándole datos en el cuerpo.
- 7.- **¿Cuál es la función de Content-Type?** Avisarle tanto al cliente como al servidor en qué formato va la información que se está intercambiando, por ejemplo, si es un JSON.
- 8.- **¿Qué significa recibir un código 200?** Que todo salió bien; la petición se procesó y se completó sin ningún problema.
- 9.- **¿Qué diferencia existe entre 200 y 201?** El 200 es un éxito general de que la consulta funcionó, mientras que el 201 específico significa que el recurso se creó correctamente (típico de un POST).
- 10.- **¿Qué significa un código 404?** Que el servidor de plano no encontró la ruta o el recurso que estabas buscando.
- 11.- **¿Qué representan los Request Headers?** Los metadatos o detalles extra que manda el cliente, como el tipo de navegador, qué formatos acepta o el tamaño de lo que envía.
- 12.- **¿Qué representan los Response Headers?** La información que el servidor te manda de regreso sobre la respuesta, como la fecha, el tipo de servidor y el formato de los datos.
- 13.- **¿Qué información contiene el cuerpo de una petición POST?** Los datos estructurados (por lo general en JSON) que quieres mandar y guardar en el servidor.
- 14.- **¿Qué significa latencia?** El tiempo total que se tarda una solicitud en ir desde el cliente hasta el servidor y regresar con la respuesta.
- 15.- **¿Qué elementos pueden incrementar el tiempo total de respuesta?** La distancia geográfica, la velocidad del internet, la resolución del DNS, el tiempo que



tarde el servidor procesando la base de datos o si hay tráfico pesado.

16.- ¿Por qué una API puede ser utilizada por diferentes clientes? Porque está desacoplada de la interfaz; mientras respeten el contrato y el protocolo HTTP, le da igual si le habla una aplicación móvil, web, Python o curl.

17.- ¿Qué ventajas tiene utilizar JSON para intercambiar información?

Que es súper ligero, fácil de leer para los humanos y compatible con casi cualquier lenguaje de programación moderno.

18.- ¿Por qué un desarrollador utilizaría Postman o curl si ya tiene un navegador?

Porque con estas herramientas puedes probar métodos avanzados (como POST, PUT, DELETE), mandar cuerpos personalizados y automatizar pruebas mucho más fáciles que en el navegador.

19.- ¿Qué relación existe entre HTTP y el modelo cliente-servidor?

HTTP es justo el idioma o el protocolo de comunicación que usan el cliente y el servidor para entenderse y pasarse los datos.

20.- ¿Qué relación existe entre esta práctica y el diseño API-first?

Que nos enseña que primero debemos definir bien el contrato y las rutas de la API antes de ponernos a hacer las pantallas o la lógica de los clientes.



Educación
Secretaría de Educación Pública



TECNOLÓGICO
NACIONAL DE MÉXICO

